Lab: Build the OrbitCo KPI Summary

Type: Hands-on lab (individual work) **Time:** 15 minutes (12 min individual work + 3 min share-out)

Objective

Extend the cleaned workbook with a Service Level Agreement (SLA) lookup table, a joined-in SLA target column, an SLAStatus computed column, and a Key Performance Indicator (KPI) summary sheet — all driven by structured-reference formulas so the workbook survives next week's data refresh without edits.

Scenario

Your director sends a follow-up: "Nice clean data. Now show me — how many tickets, what the Mean Time To Resolve (MTTR) was overall and by severity, how many SLA breaches, and what our uptime percentage was. By tomorrow." You already have `tickets_clean` from the previous lab. You also have `orbitco_uptime_raw.csv` — the downtime log from monitoring. OrbitCo's SLA targets are High = 60 minutes, Medium = 240 minutes, Low = 1440 minutes. The KPI summary will be the single sheet your director opens every Monday for the 30-day review. It must be correct when the data grows, which means every formula must use structured references, not hardcoded ranges.

Instructions

1. Open the `orbitco-workbook-[yourname].xlsx` file from the previous lab.
2. Create a new sheet named `sla_lookup`. Add two columns — `Severity` and `SLAMinutes` — with three rows: `High / 60`, `Medium / 240`, `Low / 1440`.
3. Select the data range and press **Ctrl+T**. Confirm "My table has headers," then rename the new Table to `SLALookup` on the Table Design tab.
4. Go to the `Tickets` Table on the `tickets_clean` sheet. Add a new column named `SLAMinutes` with the formula:

```
=XLOOKUP([@Severity], SLALookup[Severity], SLALookup[SLAMinutes],
"")
```

Rows with blank severity return an empty string so they do not poison downstream math.

5. Add a new column named `SLAStatus`:

```
=IF(OR([@SLAMinutes]="", [@ResolutionMinutes]=""), "Unknown",
IF([@ResolutionMinutes] > [@SLAMinutes], "BREACH", "OK"))
```

6. Import `orbitco_uptime_raw.csv` onto a sheet named `uptime_raw` using **Data → From Text/CSV**. Do not edit this sheet.

7. Copy `uptime_raw` to `uptime_clean`. Convert to a Table named `Uptime` (Ctrl+T, rename on Table Design). Confirm there is a `DowntimeMinutes` column; if downtime was exported as seconds or as a start/end pair, add a computed `DowntimeMinutes` column using the appropriate formula.

8. Create a new sheet named `outputs`. In cell A1, type the header `KPI`. In B1, type `Value`. Build the KPI table below in rows 2 through 12, using only structured references — no cell addresses like `B2:B500`:

Row	KPI (column A)	Formula (column B)
2	Total tickets (30d)	`=COUNTA(Tickets[TicketID])`
3	Tickets with missing severity	`=COUNTIF(Tickets[SeverityIsMissing], TRUE)`
4	MTTR overall (minutes)	`=AVERAGE(Tickets[ResolutionMinutes])`
5	MTTR High (minutes)	`=AVERAGEIF(Tickets[Severity], "High", Tickets[ResolutionMinutes])`
6	MTTR Medium (minutes)	`=AVERAGEIF(Tickets[Severity], "Medium", Tickets[ResolutionMinutes])`
7	MTTR Low (minutes)	`=AVERAGEIF(Tickets[Severity], "Low", Tickets[ResolutionMinutes])`
8	High-severity count	`=COUNTIF(Tickets[Severity], "High")`
9	Medium-severity count	`=COUNTIF(Tickets[Severity], "Medium")`
10	Low-severity count	`=COUNTIF(Tickets[Severity], "Low")`
11	SLA breaches	`=COUNTIF(Tickets[SLAStatus], "BREACH")`
12	Uptime percentage	`=1 - (SUM(Uptime[DowntimeMinutes]) / (30*24*60))`

9. Select cell B12 and apply percentage format with two decimal places (Home → Number format → Percentage → 2 decimals). The other MTTR cells should display as plain numbers rounded to 0 or 1 decimal for readability.

10. Go back to the `Tickets` Table. Select the entire `SLAStatus` column (click the Table column header once). Apply **Home → Conditional Formatting → Highlight Cells Rules → Equal To → `BREACH` → Light Red Fill with Dark Red Text**. The rule must

be bound to the Table column so it extends when new rows arrive — confirm this by inspecting the rule's "Applies to" field; it should read a structured reference like `=Tickets[SLAStatus]`, not a fixed range.

11. Save the file.

Example

A correct KPI table on the `outputs` sheet looks approximately like:

KPI	Value
Total tickets (30d)	413
Tickets with missing severity	19
MTTR overall (minutes)	~47
MTTR High (minutes)	~38
MTTR Medium (minutes)	~52
MTTR Low (minutes)	~180
High-severity count	~70
Medium-severity count	~210
Low-severity count	~114
SLA breaches	~40
Uptime percentage	99.90%

Exact values will vary slightly based on how you ordered the cleanup, but the shape should match: roughly 413 total, 19 flagged, uptime in the 99.8–99.95% range, and SLA breaches in the tens.

Expected Deliverables

- A workbook with five sheets plus the original `tickets_raw`: `tickets_raw` (untouched), `tickets_clean` (with an extended Tickets Table), `sla_lookup` (SLALookup Table), `uptime_raw` (untouched), `uptime_clean` (Uptime Table), and `outputs` (KPI table).
- `Tickets` Table has the original columns plus `SeverityIsMissing`, `ResolutionMinutes`, `SLAMinutes`, and `SLAStatus`.
- `outputs` sheet has eleven KPI rows, each populated with a structured-reference formula (no cell-range addresses).
- Uptime cell formatted as a percentage with two decimals.
- Conditional formatting on `SLAStatus`: any `BREACH` value highlights red-on-red automatically, and the rule's "Applies to" is the Table column (a structured reference).

Success Criteria

You have completed this lab successfully when you can:

- Read every formula on the `outputs` sheet and explain what Table column each structured reference points to.
- Paste a new row of fake ticket data at the bottom of the `Tickets` Table and confirm all eleven KPI values update automatically without opening the `outputs` sheet.
- Point at the uptime percentage and explain why the denominator is `30*24*60` (minutes in a 30-day window).
- Show that the SLA-breach conditional format extends to a newly-added row — paste a fake breach ticket and confirm the row turns red without re-applying any rule.
- Walk any KPI value on the `outputs` sheet back through the Tickets Table to the underlying rows in `tickets_raw`, proving the raw-to-processed-to-outputs lineage still holds.